

Bridge E-Hypergraphs: Specification

October 2, 2026

1 Preliminaries: Signatures and Hypergraphs

Definition 1.1 (Monoidal Signature). A (*monoidal*) *signature* Σ is given by a set of generating operations $c: n \rightarrow m$, where $n \in \mathbb{N}$ denotes the *arity* (number of inputs) and $m \in \mathbb{N}$ denotes the *co-arity* (number of outputs).

Definition 1.2 (Hypergraph). A *hypergraph* G over a signature Σ is a tuple (V, E, s, t, l) , where:

- V is a finite set of *vertices*;
- E is a finite set of *hyperedges*;
- $s: E \rightarrow V^*$ is the *source function*, mapping each hyperedge to an ordered sequence of input vertices;
- $t: E \rightarrow V^*$ is the *target function*, mapping each hyperedge to an ordered sequence of output vertices;
- $l: E \rightarrow \Sigma$ is a *labelling function* assigning each hyperedge a generator from Σ , respecting arities: for every $e \in E$, if $l(e): n \rightarrow m$, then $|s(e)| = n$ and $|t(e)| = m$.

A hypergraph is called *discrete* if $E = \emptyset$. An *unlabelled hypergraph* is a tuple (V, E, s, t) omitting the labelling function l .

2 E-Hypergraphs

The combinatorial structure of an E -hypergraph formalises hierarchical equality graphs (e -graphs) by extending hypergraphs with a hierarchy of equivalence boxes (e -boxes) and a consistency relation separating distinct equivalent components within each box.

Definition 2.1 (E-Hypergraph, Tiurin et al. [1]). An *e-hypergraph* G over a signature Σ is a tuple

$$G = (V, E, s, t, l, <, \smile),$$

where:

- (V, E, s, t) is an unlabelled hypergraph;
- $l: E \rightarrow \Sigma + 1$ is a labelling function extended with an extra distinguished value $\perp \in 1$ in its codomain; an edge $e \in E$ is called *hierarchical* (representing an e -box) if $l(e) = \perp$, and a *base edge* otherwise;
- $<$ is a strict partial order on $V + E$, called the *child relation*.

Hierarchical Notation. For any element $x \in V + E$:

- The set of *predecessors* (or *parents*) of x is denoted by

$$[x] := \{x' \in V + E \mid x' < x\}.$$

- An element $x \in V + E$ is called *top-level* if $[x] = \emptyset$.
- We write $e <^\mu x$ to denote that e is the *immediate parent* (predecessor) of x , meaning $e < x$ and there exists no $z \in V + E$ such that $e < z < x$. In this case, x is called an *immediate child* of e .
- An edge $e \in E$ is called *maximal* if $\{x' \in V + E \mid e < x'\} = \emptyset$ (i.e., it has no children in the hierarchy).

Hierarchy Axioms. The child relation $<$ must satisfy the following conditions:

- (1) **Parents are hierarchical edges:** each parent set $[x]$ contains exclusively hierarchical edges, i.e.,

$$\forall x, p \in V + E, \quad p < x \implies (p \in E \wedge l(p) = \perp);$$

- (2) **At most one immediate parent:** each element $x \in V + E$ has at most one immediate parent:

$$\forall x, p_1, p_2 \in V + E, \quad (p_1 <^\mu x \wedge p_2 <^\mu x) \implies p_1 = p_2;$$

- (3) **Maximal edges are not hierarchical:** all maximal edges carry a generator label:

$$\forall e \in E, \quad (\forall x' \in V + E, \neg(e < x')) \implies l(e) \neq \perp;$$

- (4) **Edges and incident vertices share parents:** for every hyperedge $e \in E$ and every vertex $v \in V$,

$$v \in s(e) \implies (e' <^\mu e \iff e' <^\mu v),$$

and similarly for target vertices:

$$v \in t(e) \implies (e' <^\mu e \iff e' <^\mu v).$$

Consistency Relation. The relation $\smile$ is the *consistency relation*, defined as the union of a family of equivalence relations $\smile_p$ indexed by hierarchical parent edges $p \in E$ on each set of immediate children

$$\{x \in V + E \mid p <^\mu x\},$$

satisfying:

- (a) **Closed under connectivity:** if $v \in s(e)$ or $v \in t(e)$ with $p <^\mu v$ and $p <^\mu e$, then

$$v \smile_p e;$$

- (b) **Non-triviality:** for every parent p , $\smile_p$ does not collapse into the full indiscrete relation:

$$\smile_p \neq (E_p + V_p) \times (E_p + V_p),$$

where $V_p := \{v \in V \mid p <^\mu v\}$ and $E_p := \{e \in E \mid p <^\mu e\}$. That is, whenever an e -box p has children, there exist at least two mutually inconsistent children $(x \not\smile_p y)$, ensuring distinct alternative components.

Given an e -hypergraph G , the corresponding *underlying hypergraph* G' is obtained by forgetting the hierarchy $<$ and consistency relation $\smile$.

3 The Subgraph Homomorphism Problem in E-Matching

A natural categorical intuition for defining pattern matching is to view it as a strict homomorphism. When matching a flat pattern P against a hierarchical E-hypergraph G , one might attempt to define a match as either a subgraph homomorphism of standard hypergraphs or a homomorphism in the category of E-hypergraphs. However, both approaches fundamentally fail when a pattern spans across e-box boundaries.

Consider a simple pattern P consisting of two operations A and B connected in series by a single wire (vertex) v , such that $t(A) = s(B) = (v)$. Suppose we wish to match P against a host E-hypergraph G where A belongs to an alternative within an e-box E_1 , and B belongs to an alternative within another e-box E_2 , with the two e-boxes connected in series.

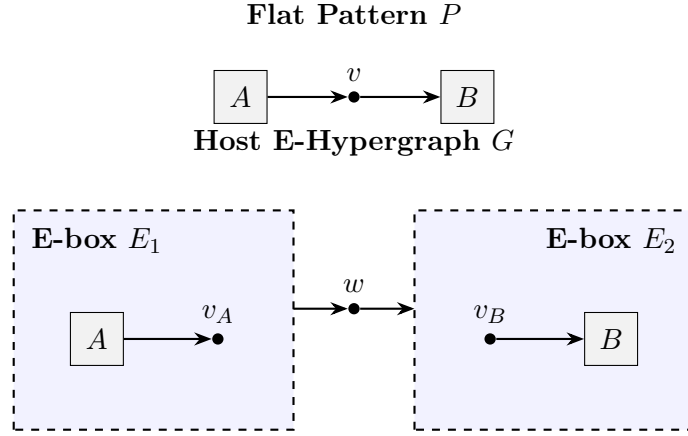


Figure 1: Topological disconnection in an E-hypergraph. In the pattern P , operations A and B directly share a vertex v . In the host E-hypergraph G , A and B are strictly isolated inside their respective e-boxes; only the e-boxes E_1 and E_2 themselves are connected via top-level vertex w .

Topological Failure of Standard Subgraph Homomorphisms. In an E-hypergraph, Axiom (4) (*Edges and incident vertices share parents*) dictates that any vertex incident to an edge must share that edge’s immediate parent:

- Any target vertex $v_A \in t(A)$ must be an immediate child of E_1 ($E_1 <^\mu v_A$).
- Any source vertex $v_B \in s(B)$ must be an immediate child of E_2 ($E_2 <^\mu v_B$).

By Axiom (2) (*At most one immediate parent*), the child vertex sets of distinct e-boxes are strictly disjoint: no vertex can be a child of both E_1 and E_2 . Furthermore, the connecting vertex w between the two e-boxes is a top-level vertex ($[w] = \emptyset$), which means neither A nor B can be incident to w .

Consequently, as illustrated in Figure 1, A and B are completely disconnected in the underlying hypergraph G' :

$$t(A) \cap s(B) = \emptyset.$$

A standard subgraph homomorphism $P \hookrightarrow G'$ requires mapping the vertex v to a single shared vertex in G' that connects A and B . Because no such shared vertex exists, a strict topological match is impossible.

Failure of Categorical E-Hypergraph Homomorphisms. To preserve the hierarchical structure, one might instead attempt to lift P to an E-hypergraph P_E that mirrors G , placing A inside a pattern e-box E_{P1} and B inside a pattern e-box E_{P2} , connected via $E_{P1} \rightarrow w \rightarrow E_{P2}$.

However, this construction fails categorically:

1. **Violation of Non-Triviality:** A pattern represents a single concrete rewrite rule without internal alternatives. Thus, E_{P_1} and E_{P_2} would each contain only a single equivalence class (the singleton operation A and B , respectively). By the *Non-triviality* condition of Definition 2.1, every e-box with children must contain at least two mutually inconsistent children. Hence, P_E is not a well-formed E-hypergraph.
2. **Loss of Pattern Generality:** Even if trivial e-boxes were permitted, structuring P to mirror the e-boxes of G would mean a pattern could only match graphs that have the exact same hierarchy decomposition, destroying the ability of a flat pattern to match across arbitrary equivalence boundaries.

Thus, matching cannot be formalized as a subgraph homomorphism (neither on the underlying hypergraph nor in the category of E-hypergraphs). Instead, an E-match is defined semantically with respect to the *flattened extractions* of G . To make this operation computable without uncompressing the graph, an explicit representation of the interface between e-boxes and their contents is required.

4 Bridge E-Hypergraphs

As demonstrated in the previous section, the child alternatives inside an e-box are strictly isolated from the parent level by Axiom (4) (*Edges and incident vertices share parents*), preventing direct topological connections between child operations and surrounding context.

To resolve this issue and make hierarchical graphs directly traversable, we introduce *Bridge E-Hypergraphs*. The core mechanism is to introduce directed *vertical hyperedges*, termed *bridge edges*, that explicitly connect the external boundary of an e-box to the internal boundary of each child alternative:

- An *input bridge* $\text{bridgeIn}(b, c)$ routes the external inputs of e-box b vertically down into the internal inputs of choice c .
- An *output bridge* $\text{bridgeOut}(b, c)$ routes the internal outputs of choice c vertically up into the external outputs of e-box b .

Remark 4.1 (Comparison with Categorical Extended Interfaces). In the foundational formulation of monoidal E-graphs by Tiurin et al. [1], the boundary between an e-box and its child contents is handled externally via categorical cospans of discrete interface graphs ($n \rightarrow n' \rightarrow G \leftarrow m' \leftarrow m$). While mathematically rigorous for DPO rewriting, this externalization splits the graph's topology between internal hyperedges (s, t) and external morphism tables $(f_{\text{int}}, g_{\text{int}})$, creating significant overhead for graph algorithms. Bridge E-Hypergraphs internalize these interface mappings directly into the hypergraph as traversable vertical edges, making the graph self-contained and uniformly traversable by standard graph matching and extraction algorithms.

4.1 Computational Structure

Separating computable data from logical invariants allows algorithms (such as pattern matching and extraction) to run on a concrete structure without incurring proof overhead.

Definition 4.2 (Edge Kinds). Given an edge set E , a signature Σ , and a set of choice identifiers $\mathcal{C}$, the set of *edge kinds* $\text{Kind}(E, \mathcal{C}, \Sigma)$ consists of:

- $\text{base}(\sigma)$ for $\sigma \in \Sigma$: standard operation edges;
- *ebox*: hierarchical equivalence boxes;

- $\text{bridgeIn}(b, c)$ for $b \in E, c \in \mathcal{C}$: an input bridge routing external inputs of e-box b to choice c ;
- $\text{bridgeOut}(b, c)$ for $b \in E, c \in \mathcal{C}$: an output bridge routing internal outputs of choice c to external outputs of e-box b .

Definition 4.3 (Bridge Graph). A *Bridge Graph* over signature Σ and choice set $\mathcal{C}$ is a tuple

$$G = (V, E, s, t, \text{kind}, \text{anchor}),$$

where:

- (V, E, s, t) is an unlabelled hypergraph;
- $\text{kind}: E \rightarrow \text{Kind}(E, \mathcal{C}, \Sigma)$ classifies each hyperedge;
- $\text{anchor}: E \times \mathcal{C} \rightarrow V + E$ maps each e-box and choice identifier to a canonical representative element (acting as a functional Union-Find root for the choice).

4.2 Bridge Graph Laws

A Bridge Graph G is called a *valid Bridge E-Hypergraph* if there exist a strict partial order $<$ (child hierarchy) and a family of consistency equivalence relations $\prec_p$ satisfying the following laws:

L1. Hierarchy Foundations:

- *Parents are E-Boxes*: If $p < x$, then $p \in E$ and $\text{kind}(p) = \text{ebox}$.
- *Unique Immediate Parent*: If $p_1 <^\mu x$ and $p_2 <^\mu x$, then $p_1 = p_2$.
- *Maximal Edges*: If an edge e has no children, then $\text{kind}(e) \neq \text{ebox}$.

L2. Scope Preservation (Bridge Exemption): For all non-bridge edges $e \in E$, every incident vertex shares the immediate parent of e :

$$\forall v \in s(e) \cup t(e), \quad (p <^\mu e \iff p <^\mu v).$$

(Bridge edges are explicitly exempt, as they span across hierarchy levels).

L3. Anchor Validity: If an e-box b has a bridge edge for choice c , the corresponding anchor is an immediate child:

$$b <^\mu \text{anchor}(b, c).$$

L4. Interface Alignment (Vertical Boundary Routing): For any bridge edge e associated with e-box b and choice c :

- If $\text{kind}(e) = \text{bridgeIn}(b, c)$, then:

$$s(e) = s(b), \quad |t(e)| = |s(b)|, \quad t(e) \text{ has no duplicates,} \quad \forall v \in t(e), \quad b <^\mu v.$$

Furthermore, $t(e)$ represents pure inputs: no child edge $e' <^\mu b$ consistent with choice c can have any vertex in $t(e)$ as a target ($t(e) \cap t(e') = \emptyset$).

- If $\text{kind}(e) = \text{bridgeOut}(b, c)$, then:

$$t(e) = t(b), \quad |s(e)| = |t(b)|, \quad s(e) \text{ has no duplicates,} \quad \forall v \in s(e), \quad b <^\mu v.$$

Furthermore, $s(e)$ represents pure outputs: no child edge $e' <^\mu b$ consistent with choice c can have any vertex in $s(e)$ as a source ($s(e) \cap s(e') = \emptyset$).

L5. Choice Consistency: All internal ports of a bridge edge are consistent with the designated anchor:

$$\begin{aligned} \forall v \in t(\text{bridgeIn}(b, c)), \quad v \smile_b \text{anchor}(b, c), \\ \forall v \in s(\text{bridgeOut}(b, c)), \quad v \smile_b \text{anchor}(b, c). \end{aligned}$$

L6. Exhaustiveness and Uniqueness: For every e-box b and every child alternative, there exists a unique bridgeIn and a unique bridgeOut edge connecting to that alternative. Distinct choices have mutually inconsistent anchors.

L7. Bridge Pairing: Every input bridge $\text{bridgeIn}(b, c)$ has a corresponding output bridge $\text{bridgeOut}(b, c)$, and vice versa.

4.3 Example: Morphism of $\text{EHypI}(\Sigma)$ vs. Bridge E-Hypergraph

To make the structural difference concrete, we examine the canonical example of an e-box with two alternatives from Tiurin et al. [1, Fig. 6], illustrated side-by-side with its equivalent Bridge E-Hypergraph in Figure 2.

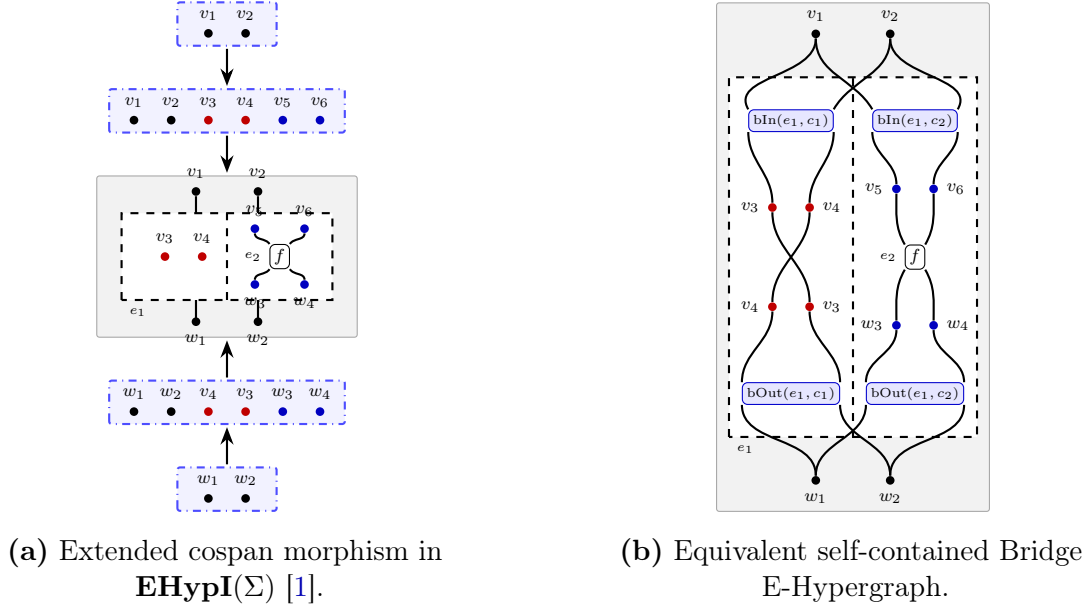


Figure 2: Visual comparison between the categorical extended interface representation of Tiurin et al. [1] (left) and the internalized Bridge E-Hypergraph representation (right).

In this example, an e-box e_1 of arity $2 \rightarrow 2$ has external input vertices (v_1, v_2) and external output vertices (w_1, w_2) , encapsulating two equivalent implementations:

- **Alternative 1 (red):** A wire permutation swapping the two ports, mapping inputs (v_3, v_4) to outputs (v_4, v_3) .
- **Alternative 2 (blue):** A generator edge e_2 labeled f , consuming inputs (v_5, v_6) and producing outputs (w_3, w_4) .

Structural Analysis of the Extended Interface Model (Figure 2a). In the original model of Tiurin et al., the graph G contains *no edges* connecting the external ports (v_1, v_2, w_1, w_2) to the internal alternatives. Inside G , both alternatives are isolated islands.

To specify how signals flow across the hierarchy boundary, the model requires an external cospan of four discrete interface hypergraphs:

$$\text{Disc}(2) \xrightarrow{f_{\text{ext}}} \text{Disc}(6) \xrightarrow{f_{\text{int}}} G \xleftarrow{g_{\text{int}}} \text{Disc}(6) \xleftarrow{g_{\text{ext}}} \text{Disc}(2).$$

The connection between external and internal ports is recorded strictly via index positions in the extended discrete interface **Disc**(6):

- f_{ext} embeds external inputs v_1, v_2 at positions 1 and 2.
- The internal inputs for Alternative 1 (v_3, v_4) occupy positions 3 and 4.
- The internal inputs for Alternative 2 (v_5, v_6) occupy positions 5 and 6.

Because this information resides in external index tables rather than in graph adjacency lists, an algorithm traversing G cannot follow pointers across the boundary without querying the inverse mappings f_{int}^{-1} and g_{int}^{-1} .

Structural Analysis of the Bridge E-Hypergraph Model (Figure 2b). In the Bridge E-Hypergraph, the four external interface graphs and their cospan mappings are completely eliminated. Instead, the interface routing is internalized as concrete, traversable hyperedges:

- Alternative 1 is equipped with an input bridge $\text{bridgeIn}(e_1, c_1)$ with $s = (v_1, v_2)$ and $t = (v_3, v_4)$, and an output bridge $\text{bridgeOut}(e_1, c_1)$ with $s = (v_4, v_3)$ and $t = (w_1, w_2)$.
- Alternative 2 is equipped with an input bridge $\text{bridgeIn}(e_1, c_2)$ with $s = (v_1, v_2)$ and $t = (v_5, v_6)$, and an output bridge $\text{bridgeOut}(e_1, c_2)$ with $s = (w_3, w_4)$ and $t = (w_1, w_2)$.

The graph is now 100% self-contained: graph matching, reachability, and extraction algorithms can traverse directly from (v_1, v_2) into either alternative and exit to (w_1, w_2) purely by following the standard source and target functions s and t .

5 The Category of Bridge Graphs

Definition 5.1 (Choice Context and Bridge Reachability). Let $G = (V, E, s, t, \text{kind}, \text{anchor})$ be a Bridge Graph over choice set $\mathcal{C}$. A *choice context* $C \subseteq E \times \mathcal{C}$ is a set of active choices such that for every e-box $b \in E$, there is at most one choice $(b, c) \in C$.

Given a choice context C , the *bridge reachability relation* $\rightsquigarrow_C$ on V is the reflexive-transitive closure of the following relations:

1. **Bridge-In traversal:** If $\text{kind}(b) = \text{bridgeIn}(\text{box}, c)$ with $(\text{box}, c) \in C$, then for each port index $i \in \{1, \dots, |s(b)|\}$:

$$s(b)[i] \rightsquigarrow_C t(b)[i].$$

2. **Bridge-Out traversal:** If $\text{kind}(b) = \text{bridgeOut}(\text{box}, c)$ with $(\text{box}, c) \in C$, then for each port index $j \in \{1, \dots, |s(b)|\}$:

$$s(b)[j] \rightsquigarrow_C t(b)[j].$$

Definition 5.2 (Bridge Morphism). Let G_1 and G_2 be Bridge Graphs. A *bridge morphism* from G_1 to G_2 is a triple $h = (h_E: E_1 \rightarrow E_2, h_V: V_1 \rightarrow V_2, C)$, where C is a consistent choice context on G_2 , such that:

- (a) **Kind Preservation:** For all $e \in E_1$, $\text{kind}_1(e) = \text{kind}_2(h_E(e))$.

- (b) **Source Reachability:** For all $e \in E_1$ and each input port $i \in \{1, \dots, |s_1(e)|\}$:

$$h_V(s_1(e)[i]) \rightsquigarrow_C s_2(h_E(e))[i].$$

- (c) **Target Reachability:** For all $e \in E_1$ and each output port $j \in \{1, \dots, |t_1(e)|\}$:

$$t_2(h_E(e))[j] \rightsquigarrow_C h_V(t_1(e)[j]).$$

Definition 5.3 (Bridge Matching). A *matching* of a flat pattern hypergraph P in a host Bridge Graph G is a bridge morphism $h = (h_E, h_V, C): P \rightarrow G$ such that h_E is injective.

Proposition 5.4. *Bridge Graphs together with Bridge Morphisms form a category **BridgeHyp**.*

References

- [1] Aleksei Tiurin, Chris Barrett, Dan R. Ghica, and Nick Hu. Equivalence Hypergraphs: DPO Rewriting for Monoidal E-Graphs. *arXiv preprint arXiv:2406.15882v2*, 2025.